

Context Shaping for 19- and 20-Round SHA-256 Compression Preimages

Kameldip Singh Basra

Independent Researcher

kameldipbasra@gmail.com

ORCID: 0009-0001-0977-4614

September 2026

Abstract

We study one-block SHA-256 compression reduced to 19 or 20 rounds, with the standard initial value, feed-forward retained, all 256 output bits fixed, and an unrestricted 512-bit input block. A global table for $u \mapsto \sigma_0(u) - u$ underlies an earlier 19-round construction. We show that choosing internal context words so that $a_4 = a_5$ and $e_8 = e_9 = 0xFFFFFFFF$ makes its three schedule constraints triangular: three table lookups replace a fixed-point iteration. The same conditions reduce the remaining consistency test to $\text{Maj}(a_4, a_3, a_2) = a_3$, which holds for exactly 3^{32} of the 2^{64} pairs (a_2, a_3) . Under an explicit lookup-distribution model this predicts one 19-round preimage per 39,124 swept values; three recorded runs give one per 38,878. A 64-target run produced 13,840 forward-verified preimages in 151.7 seconds. The often-quoted 12 ms figure is a marginal estimate obtained from measured single-core throughput and yield after a reusable 16-GiB table has been built, not a cold-start latency.

At 20 rounds, candidates must also satisfy an exact fourth schedule constraint. Existing residual tests are consistent with the working model that this condition contributes a factor 2^{-32} ; they do not prove uniformity at exact zero or rule out all alternative policies. With three stored roots per table value, the model predicts a mean cost of about $2^{45.4}$ swept values, or about 15 hours at the measured A100 production rate. We report two computed, forward-verified examples: one for a solver-generated target after 2.45 GPU-hours and one for the fixed all-ones target after 16.16 GPU-hours. The six recorded production runs total about 71.7 GPU-hours. The method combines known choice- and majority-function degeneracies. Our negative 21-round observations apply only to the tested construction and do not establish a lower bound. These results concern reduced-round compression, not padded or full SHA-256.

1 Introduction

A *preimage* of the SHA-256 compression function reduced to R rounds is a 512-bit block $W_0, \dots, W_{15}$ such that the R -round compression of the standard initial value with that block, plus the feed-forward, equals a given 256-bit digest. We retain feed-forward but impose no SHA-256 padding format on the block. Thus the object is the one-block reduced-round compression function, not the standard padded hash function. The fixed initial value also distinguishes it from a free-start or pseudo-preimage problem. All target bits are fixed; this is not a partial-output or collision problem. This is the model used by Zaikin [12] and by our earlier paper [1].

Two lines of work address this problem. The meet-in-the-middle line reaches high step counts at costs just below brute force: 43 steps at $2^{254.9}$ [5], 45 at $2^{255.5}$ [6], and, since February 2026, 47 at $2^{255.6}$ [7]. None of those attacks has been executed in full (the largest computed fragment is discussed in Section 8), and the cheapest figure anywhere in that line is 2^{240} at 24 steps [4]. The practical line, which actually computes preimages, has moved slowly: 16 rounds

by SAT solving in 2012 [8], 17 and 18 rounds for the all-zero digest in 2025 [9], and 19 rounds in January 2026, when Zaikin [12] inverted the all-ones digest in 18 h 33 min on 192 CPU cores and wrote that “it is likely that additional effective algorithmic techniques are required to invert . . . 20-round SHA-256 compression functions” ([12], §8). Our earlier paper [1] reached 19 rounds by a different, algebraic route. Its screened/control campaign gave a pooled point estimate of about two GPU-hours per success, with substantial sampling uncertainty and no events in its random-context control arm. A companion note [2] modelled the 20-round extension as a further 2^{-32} schedule filter.

This paper supersedes the performance estimates of our earlier manuscripts for this chosen-context family and reports two computed and verified preimages of the 20-round SHA-256 compression function. To our knowledge, after the dated and scoped literature search described in Section 8, no published work had exhibited such a full-output witness beyond 19 rounds.

What the base attack does. The attack of [1] fixes the last eight state words from the digest by a backward chain, chooses the seven state words $a_4, \dots, a_{10}$ at random (the *context*), sweeps a_0 over all 2^{32} values, and recovers a_1, a_2, a_3 from the three message-schedule constraints on W_{16}, W_{17}, W_{18} using a 16-GiB table that inverts $u \mapsto \sigma_0(u) - u$. Two things make it expensive. The constraints on W_{17} and W_{18} are coupled through the round function, so a_2 and a_3 are found by a fixed-point iteration that converges for roughly one C_0 survivor in 2^{16} ($1.3\text{--}1.9 \times 10^{-5}$ per survivor in [1]). And the constraint on W_{16} needs W_9 , which depends on the still-unknown a_2, a_3 ; the attack freezes them at zero, solves, and afterwards tests that the true W_9 equals the provisional one. The archived experiments measure substantial, context-dependent enrichment in this final condition; the earlier paper reports the resulting campaign rates rather than treating the residual as uniform.

What this paper does. We choose the context instead of drawing it at random. Three conditions, $a_4 = a_5$ together with $e_8 = e_9 = 0\text{x}\text{FFFFFFFF}$ (the last two each imposed by solving one round equation for one context word), have two consequences (Section 3). The coupling that forced the iteration disappears: the constraint on W_{17} loses its dependence on a_3 , so a_1, a_2, a_3 are obtained by three table lookups. And the consistency residual collapses to $\text{Maj}(a_4, a_3, a_2) - a_3$, a difference of two bitwise functions whose equality is a per-bit condition $\text{Maj}(a_4, a_3, a_2) = a_3$, satisfied by exactly 3^{32} of the 2^{64} pairs.

Results.

- At 19 rounds the cost per preimage is about $2^{15.25}$ swept a_0 , pooled over 1.14×10^9 swept values and 29,398 verified preimages in three runs. The historical random-context campaign reported $2^{38.3}$ swept values per preimage when its screened and control arms were pooled, but that mixed-arm number is not an estimate for uniformly sampled contexts; we retain it only as a description of the earlier implementation (Section 5). The model-derived value is $1/(0.633673^3 \cdot (3/4)^{32}) = 39,124$; the three-run pooled value is 38,878 (1,142,947,840 swept a_0 over 29,398 preimages), and the largest separate replication (Table 1, 24 contexts) alone gives 39,123. One CPU core produces a preimage at a marginal modelled time of about 12 ms, derived from measured throughput and yield after table construction. The all-ones digest of [12] and the all-zero benchmark of [9] each yield about a hundred preimages per second.
- At 20 rounds the fourth schedule constraint is an exact 32-bit filter in which we find no bias that a table policy or a context choice could exploit in the tested distributions and statistics (Section 6). Assuming its exact-zero rate is 2^{-32} , three-root tables and a fused GPU kernel give a modelled mean cost of $2^{45.4}$ swept a_0 , about 15 hours at the measured A100 production rate. We computed two 20-round preimages: one for a digest generated

by the solver, after 2.45 h on one A100, and one for the all-ones digest, after 16.2 h on one of two A100s searching that digest. To our knowledge, and after the search described in Section 8 (as of September 2026), no computed preimage of the SHA-256 compression function beyond 19 rounds had previously been published.

- At 21 rounds two additional exact 32-bit constraints remain. Under the explicitly stated model that their residuals are uniform and independent, they contribute a factor 2^{64} relative to 19 rounds. Measurements on 214,583 candidates and searches over a finite menu of context families are consistent with that model but do not prove it or exclude other representations (Section 7).

What is and is not new. The two levers are known. Saturating the choice function, $\text{Ch}(\text{0xFFFFFFFF}, y, z) = y$, is the “absorption property” used for message stealing by Aoki et al. [5]; the majority degeneracy $\text{Maj}(x, x, y) = x$ is standard and appears in Guo et al. [7] as an auxiliary condition that absorbs propagation through Maj (their Proposition 11 and Remark 3). Our own companion note [2] applies both levers to a different constraint and dismisses the result because one of its conditions tied a context word to an unknown, which would force the constants to be rebuilt per candidate. What is new here is the specific pairing $a_4 = a_5$, which ties two *context* words together and costs nothing per candidate, its application to the constraint that carries the iteration, and the observation that the same choice turns a 32-bit modular consistency check into a bitwise one. We have found no precedent for that last effect. The barrier argument of [2] identified the remaining residual and modelled its cost as a factor 2^{32} relative to the 19-round generator; what it did not anticipate is that the 19-round generator itself could become about 2^{23} cheaper. Section 9 reconciles the two manuscripts. The individual choice- and majority-function identities are prior art; the claimed contribution is their particular joint use here and the resulting residual factorisation.

Scope. Nineteen and twenty of SHA-256’s 64 rounds. Nothing in this paper bears on full SHA-256; on full SHA-224, which shares the compression function but uses a different initial value and truncates its output; on SHA-512, for which the table would need 2^{64} entries; or on any deployed system. The two 20-round targets are respectively image-derived and fixed all-ones. Their existence does not prove that every 256-bit target is attainable or that the modelled mean applies uniformly to every target.

2 Preliminaries

Notation. All words are 32-bit and arithmetic is modulo 2^{32} unless stated. $\text{ROTR}^n(x)$ denotes a right rotation and $\text{SHR}^n(x)$ a logical right shift. The SHA-256 functions of FIPS 180-4 [3] are

$$\begin{aligned}\Sigma_0(x) &= \text{ROTR}^2(x) \oplus \text{ROTR}^{13}(x) \oplus \text{ROTR}^{22}(x), \\ \Sigma_1(x) &= \text{ROTR}^6(x) \oplus \text{ROTR}^{11}(x) \oplus \text{ROTR}^{25}(x), \\ \sigma_0(x) &= \text{ROTR}^7(x) \oplus \text{ROTR}^{18}(x) \oplus \text{SHR}^3(x), \\ \sigma_1(x) &= \text{ROTR}^{17}(x) \oplus \text{ROTR}^{19}(x) \oplus \text{SHR}^{10}(x).\end{aligned}$$

Also, $\text{Ch}(e, f, g) = (e \wedge f) \oplus (\neg e \wedge g)$ and $\text{Maj}(a, b, c) = (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c)$. Following [1], the state after round r is written a_r and e_r , with $a_{-1}, \dots, a_{-4}$ and $e_{-1}, \dots, e_{-4}$ the eight words of the standard initial value, in order

6a09e667, bb67ae85, 3c6ef372, a54ff53a, 510e527f, 9b05688c, 1f83d9ab, 5be0cd19.

Round r consumes W_r and the standard constant K_r :

$$\begin{aligned} T_1^{(r)} &= e_{r-4} + \Sigma_1(e_{r-1}) + \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) + K_r + W_r, \\ T_2^{(r)} &= \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \end{aligned}$$

$a_r = T_1^{(r)} + T_2^{(r)}$ and $e_r = a_{r-4} + T_1^{(r)}$. Hence $e_r = a_{r-4} + a_r - T_2^{(r)}$ is determined by the a -words alone, and every message word is recovered from the state:

$$W_r = a_r - T_2^{(r)} - e_{r-4} - \Sigma_1(e_{r-1}) - \text{Ch}(e_{r-1}, e_{r-2}, e_{r-3}) - K_r. \quad (1)$$

The digest after R rounds is the initial value plus $(a_{R-1}, a_{R-2}, a_{R-3}, a_{R-4}, e_{R-1}, \dots, e_{R-4})$.

For $t \geq 16$, the message schedule is

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16}. \quad (2)$$

Backward chain from the target. Subtracting the standard initial value word by word from the target digest determines $(a_{R-1}, \dots, a_{R-4}, e_{R-1}, \dots, e_{R-4})$. For $r = R-1, R-2, R-3, R-4$, in that order, compute

$$T_2^{(r)} = \Sigma_0(a_{r-1}) + \text{Maj}(a_{r-1}, a_{r-2}, a_{r-3}), \quad T_1^{(r)} = a_r - T_2^{(r)}, \quad a_{r-4} = e_r - T_1^{(r)}. \quad (3)$$

Each step uses only words already known from the digest or an earlier step, so the target fixes eight consecutive a -words. This is an algebraic inversion of four state updates; it neither chooses nor uses a target preimage.

The base attack. Given the digest, the backward chain of [1] recovers $a_{R-8}, \dots, a_{R-1}$. At $R = 19$ the free words are $a_0, \dots, a_{10}$; the attack fixes $a_4, \dots, a_{10}$ (the context), sweeps a_0 , and must solve for a_1, a_2, a_3 against the schedule constraints

$$C_j: \quad W_{16+j} = \sigma_1(W_{14+j}) + W_{9+j} + \sigma_0(W_{1+j}) + W_j, \quad j = 0, 1, 2, \quad (4)$$

in which every W is expressed through (1). Because a_{j+1} enters W_{j+1} with coefficient $+1$ and W_{9+j} with coefficient -1 , C_j can be written as $\sigma_0(W_{j+1}) - W_{j+1} = (\text{terms free of } a_{j+1})$, and a global table inverting $u \mapsto \sigma_0(u) - u$ yields W_{j+1} , hence a_{j+1} , in one lookup. The map $u \mapsto \sigma_0(u) - u$ is not a bijection: its image has exactly 2,721,603,628 of the 2^{32} values, a fraction 0.633673, and the published table stores one root per image value, the largest.

The difficulty is that the right-hand side of C_1 depends on a_3 and that of C_2 on a_2 , so the attack iterates $C_1 \rightarrow a_2$, $C_2 \rightarrow a_3$ to a fixed point; and the right-hand side of C_0 contains W_9 , which depends on a_2, a_3 through

$$W_9 = \underbrace{T_1^{(9)} - K_9}_{\text{context}} - e_5 - \Sigma_1(e_8) - \text{Ch}(e_8, e_7, e_6), \quad e_5 = a_1 + a_5 - \Sigma_0(a_4) - \text{Maj}(a_4, a_3, a_2), \quad (5)$$

with $e_6 = a_2 + T_1^{(6)}$ and $e_7 = a_3 + T_1^{(7)}$. The attack evaluates W_9 at $a_2 = a_3 = 0$, a provisional value $\hat{W}_9$, solves C_0 with it, and finally tests

$$\varepsilon = W_9^{\text{conv}}(a_2, a_3) - W_9^{\text{conv}}(0, 0) \stackrel{?}{=} 0, \quad (6)$$

where W_9^{conv} is the a_1 -free part of (5). The measured rates in [1] are about 1.6×10^{-5} fixed points per C_0 survivor and one success of (6) per 2^{22} fixed points, giving about one preimage per 80 attempts of 2^{32} swept a_0 each, i.e. $2^{38.3}$ swept a_0 per preimage.¹

¹The pooled campaign estimate of [1], $\hat{\mu} = 3/240$ per attempt, mixes a screened arm with a control arm that produced no events, and carries a 95% interval of $[2.6 \times 10^{-3}, 3.7 \times 10^{-2}]$ for the pooled rate. It is retained here as a historical work figure for that campaign, not as a rate for uniform random contexts.

3 The Submask Context Family

Definition 1 (The family). *A context is in the submask family if*

$$a_4 = a_5 = v \quad \text{for some } v, \quad e_8 = 0\text{x}\text{FFFFFFFF}, \quad e_9 = 0\text{x}\text{FFFFFFFF}.$$

Since $e_8 = a_4 + a_8 - T_2^{(8)}$ and $e_9 = a_5 + a_9 - T_2^{(9)}$, the last two conditions are met by choosing a_6, a_7 freely and then setting $a_8 = -1 - a_4 + \Sigma_0(a_7) + \text{Maj}(a_7, a_6, a_5)$ and $a_9 = -1 - a_5 + \Sigma_0(a_8) + \text{Maj}(a_8, a_7, a_6)$. The words v, a_6, a_7, a_{10} (and a_{11} at $R = 20$) remain free, so the family contains 2^{128} contexts at 19 rounds and 2^{160} at 20.

The table solve can be written explicitly without importing notation from the earlier paper. In the order $j = 0, 1, 2$, write the round-inverted words as

$$W_{j+1} = a_{j+1} + B_j, \quad W_{9+j} = A_j - a_{j+1}, \quad (7)$$

where B_j depends only on variables already solved and A_j depends only on those variables and the context. For $j = 0$, the second relation uses the provisional $\hat{W}_9 - a_1$; for $j = 1$ it holds in the family by Proposition 1; and for $j = 2$ it follows directly from $e_7 = a_3 + T_1^{(7)}$ in (1). Substitution in (4) gives

$$\sigma_0(W_{j+1}) - W_{j+1} = W_{16+j} - \sigma_1(W_{14+j}) - W_j - A_j - B_j =: R_j. \quad (8)$$

Thus every stored root u of $\sigma_0(u) - u = R_j$ gives $W_{j+1} = u$ and $a_{j+1} = u - B_j$. This is the complete algebraic lookup used at each stage; a missing index rejects the path, and a multi-root table branches over its stored roots.

Proposition 1 (The iteration disappears). *In the submask family the right-hand side of C_1 does not depend on a_3 . Consequently, for each swept a_0, a_1, a_2 and a_3 are obtained by three successive table lookups, $C_0 \rightarrow a_1, C_1 \rightarrow a_2, C_2 \rightarrow a_3$, and every triple so obtained satisfies C_0 (with the provisional $\hat{W}_9$), C_1 and C_2 exactly.*

Proof. The only route by which a_3 reaches C_1 is through W_{10} in (4), and by (1) $W_{10} = \text{const} - e_6 - \text{Ch}(e_9, e_8, e_7)$ with $e_6 = a_2 + a_6 - \Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3)$ and $e_7 = a_3 + T_1^{(7)}$. The a_3 -dependence is therefore

$$D(a_3) = \text{Maj}(a_5, a_4, a_3) - \text{Ch}(e_9, e_8, a_3 + T_1^{(7)}).$$

With $a_5 = a_4$, $\text{Maj}(a_4, a_4, a_3) = a_4$; with $e_9 = 0\text{x}\text{FFFFFFFF}$, $\text{Ch}(e_9, e_8, \cdot) = e_8$. Both terms are constants, so D is constant. C_2 's right-hand side depends on a_2 (and on context) but its own unknown a_3 cancels as in the base attack. The three constraints are therefore triangular in (a_1, a_2, a_3) and each one-root lookup either returns its stored representative or misses; a multi-root implementation branches over the retained representatives. $\square$

Neither condition suffices alone: an isolation test on three targets gave 0–16 triangular solutions per 2^{22} swept a_0 for random contexts, 3–391 with $a_5 = a_4$ only, 2–236 with $e_9 = 0\text{x}\text{FFFFFFFF}$ only, and about 1.07×10^6 with both, i.e. 0.402 per C_0 survivor, which is 0.634^2 , the probability that the two further lookups both hit.

Proposition 2 (The consistency check collapses). *In any context with $a_4 = v$ and $e_8 = 0\text{x}\text{FFFFFFFF}$ (the condition $a_5 = a_4$ is not needed for this part),*

$$\varepsilon = \text{Maj}(v, a_3, a_2) - a_3 \pmod{2^{32}}. \quad (9)$$

With $P = a_3 \wedge \neg a_2 \wedge \neg v$ and $Q = a_2 \wedge \neg a_3 \wedge v$, one has $\text{Maj}(v, a_3, a_2) = a_3 - P + Q$: the first operation clears selected one-bits and the second sets selected zero-bits, so those two operations

respectively use no borrows and no carries. Hence $\varepsilon = 0$ if and only if $\text{Maj}(v, a_3, a_2) = a_3$ as bit strings, equivalently

$$(a_2 \oplus a_3) \wedge (a_3 \oplus v) = 0, \quad (10)$$

which holds for exactly 3^{32} of the 2^{64} pairs (a_2, a_3) , i.e. with probability $(3/4)^{32} = 1.004524 \times 10^{-4}$ for uniform independent a_2, a_3 , for every v .

Proof. With $e_8 = 0\text{xFFFFFFFF}$, $\text{Ch}(e_8, e_7, e_6) = e_7 = a_3 + T_1^{(7)}$ in (5), so the a_1 -free part of W_9 is $W_9^{\text{conv}} = \text{const} + \Sigma_0(v) - a_5 + \text{Maj}(v, a_3, a_2) - a_3 - T_1^{(7)}$. Evaluating at $a_2 = a_3 = 0$ gives the same expression with $\text{Maj}(v, 0, 0) = 0$ and $a_3 = 0$, and subtracting yields (9). For the decomposition, compare bits: where $v_i = 0$, Maj is $a_{3,i} \wedge a_{2,i}$, which differs from $a_{3,i}$ exactly when $a_{3,i} = 1, a_{2,i} = 0$, the bits of P ; where $v_i = 1$, Maj is $a_{3,i} \vee a_{2,i}$, which differs from $a_{3,i}$ exactly when $a_{3,i} = 0, a_{2,i} = 1$, the bits of Q . Since $P \subseteq a_3$, $a_3 - P$ clears bits without borrows, and since Q is disjoint from $a_3 - P$, adding Q sets bits without carries. Independently of this representation, two 32-bit words have zero modular difference exactly when they are equal. Therefore $\varepsilon = 0$ iff $P = Q = 0$, i.e. iff each bit position avoids the two forbidden patterns; the surviving pairs number 3^{32} . $\square$

At $v = 0$ condition (10) reads $a_3 \wedge \neg a_2 = 0$: a_3 is a submask of a_2 , which names the family. The exactness of Proposition 2 was checked on 2000 random contexts and pairs, and its probability on 2^{22} random pairs at $v \in \{0, 0\text{xFFFFFFFF}, 0\text{x6A09E667}\}$, by `code/submask.family.py` in the repository, which needs no table.

Corollary 1 (Yield). *Model the three lookups as returning words that are uniform and independent of v and of one another; the count 3^{32} is exact for uniform independent pairs, and Section 5 tests the model. Under it, with a single-root table of image fraction $c = 0.633673$, each swept a_0 yields $c^3 = 0.2545$ triangular solutions, of which a fraction $(3/4)^{32}$ satisfy (6). Every such triple is a 19-round preimage. The expected number of swept a_0 per 19-round preimage is therefore*

$$\frac{1}{c^3 (3/4)^{32}} = 39,124 = 2^{15.26}.$$

The last claim of the corollary, that the collapsed condition is *sufficient*, also follows from the construction. Equation (1) makes every selected state transition hold by definition. The constraints C_0, C_1, C_2 make the expanded words W_{16}, W_{17}, W_{18} agree with the words fixed by the backward chain; at 20 rounds C_3 does the same for W_{19} . Because that chain ends at the target state, a path satisfying the applicable constraints and the exact consistency check forward-evaluates to the target. As an implementation check, on 64,000 triangular solutions recomputed from their message words, C_1 and C_2 held for all, C_0 held exactly for those with $\varepsilon = 0$, and those were preimages.

Why it works, in one sentence. The base attack’s iteration converges for about one C_0 survivor in 2^{16} rather than at the 2^{-29} of a random map on a 32-bit state [2], because the feedback into C_1 passes through a bitwise select rather than through the whole round function; the family removes that feedback altogether, and the same saturation that removes it leaves the consistency residual as a difference of two bitwise functions.

4 Implementation

Tables. The published table stores one root per image value, the largest. A random target has a Poisson-like number of roots (the exact image fraction 0.633673 is slightly above $1 - 1/e = 0.632121$, and the multiplicity distribution is slightly under-dispersed), so keeping only one discards a factor $1/c = 1.578$ of the available continuations at each lookup. Building

Table 1: Nineteen-round preimages from the submask family, single-root table, CPU (numpy). “Solutions” are triangular solutions; “ $\varepsilon = 0$ ” are verified preimages.

Run	swept a_0	solutions	per survivor	$\varepsilon = 0$	a_0 /preimage
4 generated targets	16,777,216	4,270,445	0.4017	395	42,474
64 generated targets	536,870,912	136,596,846	0.4015	13,840	38,791
separate implementation A ^a	270,532,608	—	—	6,915	39,123
separate implementation B ^a	335,544,320	85,389,673	—	8,643	38,823
all-ones digest [12]	4,194,304	1,068,641	0.4020	108	38,836
all-zero digest [9]	4,194,304	1,068,359	0.4019	115	36,472

^a Twenty-four contexts, twenty-four generated targets and twelve values of v , run from separately written code in an automated review pass that used the same specification and repository evidence (see the disclosure); 6,915 of 6,915 collapsed-condition candidates verified as preimages; the second replication, 8,643 of 8,643. Derived value: 39,124. The four-target row is the run reproduced bit for bit in `reviewers/reports.json`; the other rows are in `logs/r19_big.log` and `logs/r19_fixed_targets.log`.

second- and third-root tables (first free slot, u distinct from the roots already stored) does not change the image fraction. It raises the mean number of retained roots per uniformly selected index to 0.8979 and 0.9774, and the expected triangular paths per swept a_0 to 0.7240 and 0.9338; measured values are 0.725 (`logs/smoke19_2roots.log`) and 0.9338 (`status.json` of each production run, solutions divided by swept a_0). Each table is 16 GiB; three fit an 80-GiB GPU. Extra branches also cost work: the measured CPU sweep rates were 2.80×10^5 and 2.06×10^5 swept a_0 per second for one and two roots respectively (`logs/smoke19_1root.log`, `logs/smoke19_2roots.log`); the GPU ratio was not isolated. One index in 2^{32} is lost to the sentinel (the value `0x20000000` has the single root `0xFFFFFFFF`, which encodes the “miss” sentinel); it is documented and irrelevant to any rate.

Kernel. A first GPU implementation in eager PyTorch reached 4.47×10^8 swept a_0 per second on an H100 and 2.54×10^8 on an A100, bound not by the table gathers but by streaming 64-bit intermediates through memory. A fused Triton kernel that keeps the whole chain in 32-bit registers, gathers only from the three tables, and records only the winning a_0 for host-side reconstruction reaches $9.1\text{--}9.3 \times 10^8$ per second kernel-timed on an A100 (`FLEET.txt`), roughly $3.6\times$ the eager kernel on the same GPU, and sustained $8.6\text{--}9.0 \times 10^8$ in production including host overhead (`status.json` of each Triton pod, swept a_0 divided by elapsed time), i.e. 4.7–5.0 s per context of 2^{32} swept a_0 . Averaged over each pod it produces 402,880–402,910 collapsed-condition candidates per context (`status.json`, candidates divided by contexts), against $0.93384 \cdot (3/4)^{32} \cdot 2^{32} = 402,897$ predicted from the exact three-table yield 0.977443^3 . Two implementation traps are worth recording: Triton’s right shift is arithmetic, so every rotation must mask, and a 32-bit table index sign-extends into an illegal address unless masked to 64 bits. The kernel was validated against the reference implementation on planted preimages before use (Section 6.4).

Reference implementation and verification. Every reported preimage is rebuilt from its recovered state into sixteen message words and re-hashed by an implementation independent of the solver before it is reported, and again by the standalone verifier `code/verify_r19.py` (standard library only, no shared code with the attack), which prints `result: OK`.

5 Nineteen Rounds

Table 1 summarises the 19-round measurements. The 64-target run took 151.7 s of wall time in one unpinned NumPy process (3.5×10^6 swept a_0 per wall-second); pinned to a single core the rate is 3.4×10^6 swept a_0 per core-second (`reviewers/reports.json`, `taskset` with

Table 2: Nineteen-round preimages of the SHA-256 compression function, standard IV, all 256 bits fixed. Costs are per preimage.

	Zaikin [12]	[1]	this work
Targets	one fixed (all-ones)	six generated; one fixed	generated and fixed benchmarks
Method	tuned Kissat inside Cube-and-Conquer	random context, fixed-point iteration	submask family, three lookups
Cost	18 h 33 min on 192 cores ^a	$2^{38.3}$ swept a_0 (pooled mixed-arm campaign); about 2 GPU-hours	$2^{15.26}$ swept a_0 ; about 12 ms marginal CPU time
Preimages verified	1	7	more than 20,000 ^b

^a About 3,560 core-hours for the reported successful run, obtained from an intermediate problem whose solutions are 19-round preimages with probability $1/8$. This single duration is not an estimate of a mean. Direct attempts on the 19-round instance found nothing in seven days on 192 cores (Zaikin’s account). ^b Counts are in the logs; verified blocks with their targets are printed in `logs/r19_scale.log` (33), `logs/r19_fixed_targets.log` (4), `logs/r19_variants.log` and `logs/r19_degenerate.log` (2 each) and the `logs/smoke19_*.log` files.

OMP_NUM_THREADS=1). Combining this throughput with the measured yield gives a marginal estimate of about 12 ms of one core per preimage after the reusable table has been built; it is not an observed cold-start latency. The measured collapse rate, 1.0132×10^{-4} per solution on the 64-target run and $1.0122 \times 10^{-4} \pm 1.1 \times 10^{-6}$ on a second, larger reviewer replication (3.36×10^8 swept a_0), agrees with $(3/4)^{32} = 1.0045 \times 10^{-4}$; an absolute yield audit on a separate corpus (4.70×10^9 swept a_0 over 100 contexts at 19 and 20 rounds, 119,948 collapsed-condition candidates, every 19-round one a verified preimage), observed divided by the prediction of Corollary 1 with the measured table fill 0.63375, gives 0.9986 ± 0.0029 , so any hidden gain or loss lies within a hundredth of a bit. No material target effect is resolved in these samples: the all-ones and all-zero benchmark digests have observed yields near those of the generated targets.

Comparison. Table 2 keeps unlike hardware and cost measures explicit. Two cautions apply. The new cost is a *marginal* cost: both our attacks presuppose the same one-time 16-GiB table, which takes about 215 recorded CPU core-seconds to build, so a cold-start cost for a single preimage is dominated by the table. And the baseline of [1] is a mixed screened/control campaign rather than a hardware-matched benchmark. Its pooled work figure, $2^{38.3}$ swept a_0 , divided by the present measured $2^{15.25}$, is a work-count ratio of about 2^{23} for those two implementations. We do not turn the CPU and GPU timings in Table 2 into a controlled speed ratio. Zaikin’s 18 h 33 min is the duration of one successful run; the stated one-in-eight acceptance probability of its intermediate solutions does not by itself establish an eightfold mean runtime.

6 Twenty Rounds

At $R = 20$ the backward chain fixes $a_{12}, \dots, a_{19}$, the context is $a_4, \dots, a_{11}$, and a fourth constraint C_3 on W_{19} appears. The submask family transplants unchanged: Propositions 1 and 2 involve only $a_4, \dots, a_9$ and W_9 , and the measured rates at 20 rounds are 0.4015 solutions per C_0 survivor on 40 targets of 2^{26} swept a_0 each (`logs/c3_uniformity.log`) and a collapse rate of 1.0049×10^{-4} per solution on 8.4×10^{10} swept a_0 (`logs/c3_scaling.log`). Everything therefore reduces to the residual of C_3 ,

$$c_3 = \sigma_0(W_4) + W_3 - \kappa_3, \quad \kappa_3 = W_{19} - \sigma_1(W_{17}) - W_{12}, \quad (11)$$

which must vanish, where W_3, W_4 depend on $a_0, \dots, a_4$ and κ_3 on the context and digest only.

6.1 Measured behaviour of the fourth residual

We tested the distribution of c_3 on candidates that satisfy everything else.

- A chi-squared test on 68,414 candidates, on the low and the high 4, 6, 8, 10 and 12 bits: every $|z| \leq 2.1$ (largest: high 10 bits, $z = +2.10$).
- A scaling ladder on 8.4×10^{10} swept a_0 (2.14×10^6 candidates): the number with the low k bits of c_3 zero, against the uniform prediction, has ratio 1.01 at $k = 8$, 0.96 at 12, 1.04 at 16, 0.86 at 18 (7 against 8.2).
- The GPU campaign’s own counters (`gpu_run_snapshot/h100_status.json`, 2.96×10^9 candidates): the counts n_k are nested. Under the uniform-residual null, n_{k+1} is conditionally Binomial($n_k, \frac{1}{2}$), but the observed ratios do not thereby become unconditionally independent because their denominators are random. A score formed from the conditional transitions over bits 20–28 gives 2,654 successes of 5,370 opportunities on the H100 (expected 2,685, $z = -0.85$). The most extreme individual transition, selected after examining the eight steps, has exact unadjusted one-sided $p = 0.00843$ and Bonferroni-adjusted $p = 0.0674$. The two A100 runs give $z = +1.47$ and $z = +1.71$ over the same transitions. These are descriptive large-sample score calculations, not independent-binomial replications.
- A dedicated test on the all-ones digest itself, against generated digests as a matched control (`logs/ones_obstruction.log`; 3.95×10^6 candidates in each arm): no deficit that deepens with k appears in either arm: every cell of the all-ones arm from 8 bits on is within 1.3σ of expectation below 18 bits, and the only cell outside 2.2σ on the low side anywhere is the control arm’s 6-bit count (61,030 against 61,786, $z = -3.0$, a 1.2% shortfall absent at 8 bits and beyond). The all-ones arm runs high between 18 and 20 bits (at 20 bits, 11 observed against 3.8 expected, $z = +3.7$; conditional halving over steps 16–21 $z = +3.3$) and returns to expectation at 22 (1 against 0.9); the statistic was examined after each of six rounds with a widening window, so we read the surplus as noise and the test as excluding only an obstruction, which is what it was built for.

The deepest cell with useful occupancy is the 24-bit cell (174 candidates against 176.5 expected on the H100 run). Counts at 28 bits are too sparse to bound an atom at exact zero without distributional assumptions. Thus $P(c_3 = 0) = 2^{-32}$ is a working extrapolation, not a measured exact-zero rate. The two observed hits in Section 6.3 are consistent with it but do not estimate it precisely.

Why we find no steering. For a fixed candidate $(a_0, \dots, a_3)$ the digest and the context words $a_5, \dots, a_{11}$ enter (11) only through the additive constant κ_3 ; the word $v = a_4$ enters W_4 additively before σ_0 , and the context constants also determine which (a_1, a_2, a_3) the three lookups return. Writing $S = \sigma_0(W_4) + W_3$, a sum modulo 2^{32} , the constant contributes only a phase, $\varphi_{c_3}(\beta) = \varphi_S(\beta) e^{-2\pi i \beta \kappa_3 / 2^{32}}$, and if W_3 and $\sigma_0(W_4)$ were independent φ_S would factorise as $\varphi_{W_3} \varphi_{\sigma_0(W_4)}$; the two share a_1, a_2, a_3 , so we treat the factorisation as a model and test it on 40 contexts of about 1,700 candidates each. The largest-root policy gives W_3 a visible first harmonic, $|\varphi_{W_3}(1)| = 0.155$, while the observed $|\varphi_{\sigma_0(W_4)}(1)| = 0.025$ and $|\varphi_{c_3}(1)| = 0.026$ are near the per-context sampling scale $1/\sqrt{n} = 0.024$. An 8.4×10^6 -frequency scan found no statistic above its uniform-model reference maximum; the same scan detects the known W_3 bias. Separate mutual-information tests against a_0 , v , context identifiers and bit fields, and random linear approximations from a_0 , likewise produced maxima comparable with their shuffle or noise controls.

These tests concern the sampled context distribution, the implemented root policies and the statistics just named. They found no usable steering signal at their detection limits. Marginal

Table 3: The recorded 20-round production runs. Every run built three root tables on its device. “Model mean” is the number of collapsed candidates times the assumed 2^{-32} fourth-constraint rate.

GPU	kernel	target	contexts	swept a_0	model mean	outcome
H100	PyTorch	generated digest	7,351	3.16×10^{13}	0.690	none
A100	PyTorch	generated digest	1,839	7.90×10^{12}	0.173	none
A100	PyTorch	generated digest	1,836	7.89×10^{12}	0.172	none
A100	Triton	generated digest	1,765	7.58×10^{12}	0.166	preimage , 2.45 h
A100	Triton	all-ones	12,203	5.24×10^{13}	1.145	preimage , 16.16 h
A100	Triton	all-ones	12,075	5.19×10^{13}	1.133	none (stopped)
total				1.59×10^{14}	3.48	2 preimages

and pairwise tests do not establish the joint factorisation above, exclude sparse exact-zero effects, or rule out an unexamined table policy or context family.

6.2 Cost

Under the working assumptions that the retained lookup branches follow their measured mean, the collapsed pairs have the uniform-pair rate of Proposition 2, and $c_3 = 0$ has probability 2^{-32} , the probability of a 20-round preimage per swept a_0 is $0.9337 \cdot (3/4)^{32} \cdot 2^{-32} = 2.18 \times 10^{-14}$ with three-root tables, i.e. $2^{45.4}$ swept a_0 per preimage ($2^{47.3}$ with one root). At 8.5×10^8 per second on an A100 this model gives a mean of about 15 h. If successive candidates have a constant, independent success probability, the waiting-time model is approximately exponential; that is an additional modelling assumption, not a consequence of the two observed successes. We do not extrapolate an H100 time because the fused production kernel was not run on that device.

6.3 The computed preimages

Table 3 lists every main production run (omitting about 48 contexts, 2×10^{11} swept a_0 , on generated targets from the two all-ones runs before they were retargeted; **FLEET.txt**). The six status records sum to 71.7 GPU-hours of solving time. They produced two preimages against a model mean of 3.48. Because successful jobs stopped on a hit and related jobs were then stopped manually, this campaign was not a fixed-exposure experiment; we therefore report the comparison descriptively and do not attach a Poisson significance test to it.

Preimage of a solver-generated digest. The target is the 20-round digest of a random 55-byte message drawn by the solver, so a preimage was known to exist; the block found is a different one.

```
target: d962ca30 635f9b74 ac6c8c12 43a1a9cf 800e81bc 05f1d2e4 0764c68c 795f7388
block:  a36f4238 f2c9204e b5b6653b 070401f7 5928e4f3 fe766be2 52026907 f7ee1812
       01344603 ea505012 86cbe6cb 6ce73d0b 5c91de6a 43355e3b ff3d5e88 c1ad0b54
```

Found 2026-09-04 in context 1,765 after 2.45 h on one A100 (7.58×10^{12} swept a_0). The recovered state has $a_4 = a_5 = 0x799f3424$, $e_8 = e_9 = 0xFFFFFFFF$ and satisfies (10).

Preimage of the all-ones digest. Target **ff**...**ff** (256 one-bits), the digest Zaikin [12] inverted at 19 rounds and chose because, for an irregular output, “it may be hard to justify this choice and prove that a random pair of input and output was not picked” ([12], §5.1); no 20-round preimage of it was known to exist:

```
0b0187bf b9aea692 b66effa5 087dca3a 0caea827 1d2f9916 739a224e a87c0eae
7f9ef4a7 b318a7de a8848c61 a7a141a4 11ac114b 952299be 97aaa67f c6acfe57
```

Found 2026-09-05 in context 12,203 after 16.16 h on one A100 (5.24×10^{13} swept a_0); a second A100 on the same digest swept 5.19×10^{13} in 12,075 contexts without a hit before being stopped. The recovered state has $a_4 = a_5 = 0x19eb97f7$, $e_8 = e_9 = 0xFFFFFFFF$ and satisfies (10).

Both blocks verify with `code/verify_r19.py --rounds 20`, fail with `--rounds 19` and `--rounds 21`, and reproduce the digest under a separately written from-scratch SHA-256. Appendix A gives the commands.

The same digest, one more round. Zaikin inverted the all-ones digest at 19 rounds in 18 h 33 min on 192 CPU cores. We use the same fixed target at 20 rounds, but the unlike algorithms, hardware and single successful observations are not a controlled performance comparison.

6.4 Validation of the pipeline

Because no computed 20-round preimage was available to us before the campaign, the fourth-constraint code path had never been exercised on a genuine input, and an error in it would leave the residual looking uniform while silently discarding real hits. We therefore *planted* 20-round preimages: blocks constructed so that their internal state lies in the family and satisfies (10), then hashed for 20 rounds. Run over a window containing the true a_0 , the reference implementation found every plant whose table roots were stored (10 of 16 with two tables; a separate automated replication found 54 of 200 with one table and 145 of 200 with two, against the expected recovery fractions $c^3 = 0.25$ and $0.8979^3 = 0.72$) and never a wrong one. The Triton kernel was then checked against the reference on 12 plants with two and three tables, agreeing on every hit, and an interpreter-mode execution compared all 27 root-combination paths against the reference on identical inputs.

7 Twenty-One-Round Limitations

Each round beyond 19 adds one schedule constraint. The companion note [2] found that, for its tested random-context parameterisations, absorbing a fifth unknown creates a dependency cycle whose largest measured edge is $a_4 \rightarrow C_0$ through $\Sigma_0(a_4)$ in W_9 . The submask family does not cancel that edge; it removes a different dependency ($a_3 \rightarrow C_1$) and tests C_3 directly. We report the finite searches below as limitations of the present construction, not as an impossibility or optimality result.

Context families tested. The saturation mechanism of Proposition 1 clears a dependency when it passes through Maj or Ch with a context word among the arguments. It supplies no direct lever for a dependency through Σ_0 or σ_0 of the unknown itself. For C_3 to absorb a_4 in this triangular arrangement one would need C_1 free of a_3 and of a_4 , and C_2 free of a_4 . Table 4 reports, for six condition families, which of these hold, measured by perturbing each unknown and observing the constraint targets.

The middle column remains “no” in these six families. With $a_5 = a_4$ the a_4 -dependence of C_1 is $\Sigma_0(a_4)$ inside $T_1^{(6)}$; without it, a_3 leaks back in through $\text{Maj}(a_5, a_4, a_3)$. A symbolic search over 1,485 context-condition families from the implemented grammar (`analysis/scripts/symdep/`) found none that clears both routes. The collapse of Proposition 2 survives the tighter families (collapse rates 1.016×10^{-4} and 0.987×10^{-4} , `logs/frame_search.log`), but they gave no gain under the tested solve order.

A recurring heavy edge. The broader finite search localises the failure within its representation. Two recurring facts were observed at 20 rounds (`analysis/elimination_search/`). First, an *offset law*: every state word a_k enters the recovered message word W_{k+5} through $\Sigma_0(a_k)$,

Table 4: Observed dependency removals for six tested context families at 20 rounds (40 random instances each, measured by `scripts/frame_search2.py`; output in `logs/frame_search.log`).

Context conditions	C_1 free of a_3	C_1 free of a_4	C_2 free of a_4
random	no	no	no
$a_4 = a_5, e_8 = e_9 = -1$ (this family)	yes	no	no
$\dots$ and $a_6 = a_5$	yes	no	no
$a_5 = a_6, e_9 = e_{10} = -1$ (shifted)	no	no	yes
$a_4 = a_5 = a_6, e_8 = e_9 = e_{10} = -1$	yes	no	yes
$a_4 = \dots = a_7, e_8 = \dots = e_{11} = -1$	yes	no	yes

carried by the term $e_{k+1} = a_{k-3} + a_{k+1} - \Sigma_0(a_k) - \text{Maj}(a_k, a_{k-1}, a_{k-2})$ subtracted inside (1), and through $\Sigma_1(e_{k+4})$ with $e_{k+4} = a_k + \text{const}$; both are one-input functions of a_k itself, so the tested saturation rule does not apply to them. Second, a_1, a_2 and a_3 had fixed roles in every one of 11,086 legal context configurations examined, a_1 is heavy in C_1, C_2 and C_3 and can be absorbed only by C_0 , and likewise a_2 only by C_1 and a_3 only by C_2 . Within this arrangement, a fourth absorbed word would therefore need to be light in C_0, C_1 and C_2 and absorbable by C_3 , and the offset law puts $\Sigma_0(a_k)$ heavily into at least one of C_0, C_1, C_2 for every $k = 4, \dots, 11$.

The closest miss is a_5 , and it localises the barrier to one wire. With $a_6 = \neg a_4, a_7 = a_6$ and $e_8 = 0\text{x}\text{FFFFFFF}$ the word a_5 is *exactly* absent from C_0 and from C_2 , and with $e_{11} = 0$ it enters C_3 exactly linearly; every path but one is cut. The one that remains is $a_5 \rightarrow C_1$ through $-\Sigma_0(a_5)$ inside $e_6 = a_2 + a_6 - \Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3)$ in W_{10} . With $a_0, \dots, a_3$ fixed, the C_1 lookup target depends on a_5 through exactly four atoms, $-\Sigma_0(a_5) - \text{Maj}(a_5, a_4, a_3) + \Sigma_1(a_5 + c_9) + \text{Ch}(a_5 + c_9, e_8, e_7)$, where $c_9 = a_9 - T_2^{(9)}$ is the only context lever besides flattening the Maj and Ch terms (worth one to two bits). The kernel $\Sigma_1(x + c_9) - \Sigma_0(x)$ changes at least 10.3 bits per flipped bit of x for every c_9 in a scan of 201,058 values including every sparse one. A falsification test over about 1.6×10^6 legal context conditions outside the standard menu (arbitrary constants for $e_8, \dots, e_{11}$ and the context words; rotational, XOR, additive, Σ and GF(2)-linear ties; a symbolic search for a cancelling second occurrence; a full 2^{32} scan of the frozen- a_5 residual; and a reduced-width model exhaustive over all 2^{25} contexts at five bits and checked at 6, 8, 12 and 32) found a global minimum of 10.86 bits per flipped bit, against the 2-bit threshold at which absorption becomes possible and a control of 12.3–13.1 bits on the $a_4 \rightarrow C_0$ edge; the frozen residual had random-map-like measured collision statistics (empirical zero rate $2 \cdot 2^{-32}$ and no detected affine output bit) (`analysis/elimination_search/falsify_a5/`). The obstruction to a fourth absorption in these experiments is thus localised to this single edge. It is heavy for the same reason as the $a_4 \rightarrow C_0$ edge of [2]: Σ_0 of the unknown itself. This is a negative result for the stated representation, condition grammar and solve order, not for all table-based or algebraic methods.

Joint-residual measurement. On 214,583 freshly generated 21-round candidates (84 targets, 8.4×10^9 swept a_0 ; `analysis/scripts/symdep/gen.log`) satisfying everything up to the collapsed condition, the residuals of C_3 (at W_{19}) and C_4 (at W_{20}) were individually consistent with the tested uniformity statistics, and the estimated pairwise mutual information was below 10^{-5} bits. This does not prove joint independence or bound sparse dependence at exact zero. If both residuals are modelled as independent uniform 32-bit words, 21 rounds adds a factor 2^{64} relative to the 19-round candidate generator (or 2^{32} relative to the 20-round model), for about 2^{77} swept a_0 with three-root tables. This is a conditional estimate, not a lower bound.

Exploratory corpus checks. We mined 51,268 verified 19-round preimages and 68,680 20-round candidates for structure not forced by the construction: per-bit biases, GF(2)-linear and affine relations, context effects, neighbour structure in a_0 , residue-class structure in c_3 , and the

entropy accounting of Proposition 2. The 32 per-position marginal entropies of (a_2, a_3) sum to 50.7189 bits against the predicted $32 \log_2 3 = 50.7188$ (each position 1.58496 ± 0.00003), and the 496 estimated pairwise mutual informations between positions sum to 0.001 bits. A sum of marginal entropies is not the joint entropy, and pairwise tests do not exclude higher-order dependence; these numbers show only agreement at first and second order. The GF(2) rank deficit is exactly the three carry-free least-significant bit relations the construction implies; and no other test departed from its reference distribution. Seven separate automated analyses (see the disclosure) proposed eight candidate levers, and a separate refutation pass rejected every one within the tested representations. One methodological note: a corpus that draws a_0 with replacement contains duplicate rows that produce a spurious 69σ “collision” signal in c_3 until deduplicated.

8 Prior Work and Priority

Computed preimages of round-reduced SHA-256. 16 rounds by SAT in 2012 [8]; 17 and 18 rounds for the all-zero digest, 19 only in a weakened form (at most 31 of the 256 digest bits fixed, to zero: 32 instances at one-bit steps, all but the 32-bit one solved), and 19-round collisions, by Davydov et al. [9]; 19 rounds for the all-ones digest by Zaikin [12], extending [10, 11]; and seven target-only 19-round examples from our earlier construction, six for image-derived targets and one for the same all-ones target [1]. The repository records four automated search passes, completed in September 2026, over the IACR ePrint archive and proceedings, dblp, OpenAlex, arXiv, selected SAT, CP and AI venues, relevant repositories, and some Russian- and Chinese-language sources. Those searches found no prior published, computed witness at 20 or more rounds in the same model: standard IV, feed-forward, one unrestricted block and all 256 output bits fixed. This is a dated literature finding, not a proof of priority. The search did not systematically enumerate CP 2026, IJCAI 2026, AAAI 2026, doctoral theses, or SAT-competition logs.

Theoretical preimage attacks. The meet-in-the-middle line [4, 5, 6] reaches 45 steps at $2^{255.5}$; Guo et al. [7] extend it to 46 steps at $2^{254.5}$ and 47 at $2^{255.6}$ (pseudo-preimages at $2^{253.2}$), improve 43–45 steps, take SHA-512 to 51 steps, and describe the preceding decade as stagnant. None of these attacks has been executed; their preimages are obtained from pseudo-preimages by the generic conversion and pay for padding. Rounds and steps coincide for SHA-256 in both literatures, so no prior result becomes twenty under a different convention. The citation runs one way: the SAT line quotes the meet-in-the-middle bounds [12, 9], but [7] contains no SAT-based preimage work.

Computed partial results that must not be conflated. The artefact accompanying [7] (the repository github.com/desert-oasis/dual-sync-MITM, directory `SHA256-44_partial_pseudo_preimage`) computes messages for 44-step SHA-256 whose output has 32 or 40 of 256 bits fixed to zero, from a free chaining value: weakened on three axes at once. Bouam et al. [13] exhibit three messages whose *full* SHA-256 digests XOR to zero on their first 128 bits (a 3XOR), a brute-force computation of at least 8.7×10^{22} integer operations by the authors’ count; that is a collision-type result on half the output, not a preimage of any bits. The GraphBench benchmark [16], building on the encodings of [14, 15], emits instances labelled as SHA-256 preimages at 16–30 rounds, but with up to 384 of the 512 input bits fixed, and no round count has been claimed from them.

The levers. Choice-function absorption is the message-stealing tool of [5]; majority degeneracy is formalised in [7]; our companion note [2] records both on a different constraint. The

pairing of two context words, the application to the iteration-carrying constraint, and the bitwise collapse of the modular consistency residual are, to our knowledge, new.

9 Relation to Our Earlier Papers

This paper supersedes the cost figures of [1] and the headline of [2], and we say so precisely rather than silently.

- [1] reports about one preimage per 80 attempts of 2^{32} swept a_0 , i.e. about $2^{38.3}$ swept a_0 , when the screened and control arms of its campaign are pooled. All three events were in the screened arm; the control arm had none. The figure describes that mixed campaign, not uniformly sampled contexts. The submask family is a distinct construction with the same table and backward chain, measured at $2^{15.25}$.
- [2] estimates a factor 2^{32} for the fourth constraint under a uniform-residual model and locates the tested dependency in $\Sigma_0(a_4)$ inside W_9 . The exact residual equation and the finite negative searches remain useful; their extrapolation is a model, not a bound (Section 7). What changes is the per-context cost of the first three constraints, which the family reduces from a lossy iteration to three lookups. The note’s own caveat, that its negative results are properties of the parameterisations tried, is exactly what applied: its section on a context family that removes a_2 from C_0 applies the same two levers to C_0 and records the result as a structural observation rather than an attack step, because the Maj condition it chose tied a context word to an unknown; applying them to C_1 with a pairing on two context words is the step it did not take.
- The screening campaign of [1] reports a $3.41\times$ lift in a downstream counter. That figure is trajectory-counted; duplicates are 0.03% of converged trajectories, and identity-level deduplication can move it only within $[2.42\times, 3.44\times]$. It is now of historical interest only.

10 Conclusion

Imposing three conditions on the context words removes the iteration from the 19-round construction and reduces its final consistency check to a bitwise condition. The measured work is $2^{15.25}$ swept values per preimage, within one per cent of the yield model; the corresponding 12-ms CPU figure is a marginal estimate after reusable preprocessing. At 20 rounds the exact fourth constraint remains. Modelling its residual as uniform gives $2^{45.4}$ swept values and about fifteen A100 hours per preimage; two computed examples are exhibited, including one for the all-ones target. The six production records total 71.7 GPU-hours, while the two successful runs took 2.45 and 16.16 hours.

To our knowledge, as of September 2026 and after the search described in Section 8, no computed preimage of the SHA-256 compression function beyond 19 rounds had previously been published in the same full-output, fixed-IV model. At 21 rounds the tested residuals and context families support, but do not prove, an independent-uniform model for two 32-bit constraints. No impossibility or lower-bound claim follows. Full and padded SHA-256 are unaffected.

Acknowledgements and disclosure

The derivations, code, experiments, verification and text of this paper were developed with substantial assistance from Anthropic’s Claude, used through Claude Code under the author’s direction, including experiment design, code, analysis and drafting. OpenAI Codex assisted

with the present evidence audit, claim qualification and manuscript revision. The supplied blocks were forward-checked by the standalone verifier and by several separately written SHA-256 implementations. Other automated review sessions variously reimplemented parts of the construction or inspected shared logs and code; their reports are evidence of those specific checks, not human peer review or blanket independent replication. The artefacts are listed in Appendix B.

References

- [1] K. S. Basra. A practical preimage attack on the 19-round SHA-256 compression function via a global σ_0 -difference table. Manuscript, 2026. Source, code and witnesses at <https://github.com/kamb-code/sha256-r19-preimage>.
- [2] K. S. Basra. The 20-round barrier for table-based preimage attacks on SHA-256. Companion note, 2026.
- [3] National Institute of Standards and Technology. *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.
- [4] T. Isobe and K. Shibutani. Preimage attacks on reduced Tiger and SHA-2. In *Fast Software Encryption (FSE 2009)*, LNCS 5665, pp. 139–155. Springer, 2009.
- [5] K. Aoki, J. Guo, K. Matusiewicz, Y. Sasaki, and L. Wang. Preimages for step-reduced SHA-2. In *Advances in Cryptology – ASIACRYPT 2009*, LNCS 5912, pp. 578–597. Springer, 2009.
- [6] D. Khovratovich, C. Rechberger, and A. Savelieva. Bicliques for preimages: attacks on Skein-512 and the SHA-2 family. In *Fast Software Encryption (FSE 2012)*, LNCS 7549, pp. 244–263. Springer, 2012.
- [7] J. Guo, H. Li, M. Liu, S. Wang, and T. Zhang. Dual-syncopation meet-in-the-middle attacks: new results on SHA-2 and MD5. In *Advances in Cryptology – EUROCRYPT 2026*, LNCS 16546, pp. 121–151. Springer, 2026. DOI 10.1007/978-3-032-25333-0_5. Cryptology ePrint Archive, Report 2026/353.
- [8] E. Homsirikamol, P. Morawiecki, M. Rogawski, and M. Srebrny. Security margin evaluation of SHA-3 contest finalists through SAT-based attacks. In *Computer Information Systems and Industrial Management (CISIM 2012)*, LNCS 7564, pp. 56–67. Springer, 2012. DOI 10.1007/978-3-642-33260-9_4.
- [9] V. V. Davydov, M. D. Pikhtovnikov, A. P. Kiryanova, and O. S. Zaikin. Analysis of the cryptographic strength of the SHA-256 hash function using the SAT approach. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics*, 25(3):428–437, 2025. DOI 10.17586/2226-1494-2025-25-3-428-437.
- [10] O. Zaikin. Inverting step-reduced SHA-1 and MD5 by parameterized SAT solvers. In *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, LIPIcs 307, pp. 31:1–31:19, 2024. DOI 10.4230/LIPIcs.CP.2024.31.
- [11] O. Zaikin. Inverting cryptographic hash functions via Cube-and-Conquer. *Journal of Artificial Intelligence Research*, 81:359–399, 2024. DOI 10.1613/jair.1.15244.
- [12] O. Zaikin. Preimage attacks on round-reduced MD5, SHA-1, and SHA-256 using parameterized SAT solver. *Constraints*, 2026 (received 28 March 2025, accepted 16 December 2025, published online 23 January 2026). DOI 10.1007/s10601-025-09383-0.

- [13] M. Bouam, C. Bouillaguet, C. Delaplace, and C. Noûs. Computational records with aging hardware: controlling half the output of SHA-256. *Parallel Computing*, 106:102804, 2021. DOI 10.1016/j.parco.2021.102804. Cryptology ePrint Archive, Report 2021/886.
- [14] S. Nejati, J. H. Liang, C. Gebotys, K. Czarnecki, and V. Ganesh. Adaptive restart and CEGAR-based solver for inverting cryptographic hash functions. In *Verified Software: Theories, Tools, and Experiments (VSTTE 2017)*, LNCS 10712, pp. 120–131. Springer, 2017. DOI 10.1007/978-3-319-72308-2_8.
- [15] N. Alamgir, S. Nejati, and C. Bright. SHA-256 collision attack with programmatic SAT. In *Joint Proceedings of the 9th Workshop on Practical Aspects of Automated Reasoning (PAAR) and the 9th Satisfiability Checking and Symbolic Computation Workshop (SC-Square)*, CEUR Workshop Proceedings vol. 3717, pp. 91–110, 2024.
- [16] T. Stoll, C. Qian, B. Finkelshtein, A. Parviz, D. Weber, F. Frasca, H. Shavit, A. Siraudin, A. Mielke, M. Anastacio, E. Müller, M. Bechler-Speicher, M. Bronstein, M. Galkin, H. Hoos, M. Niepert, B. Perozzi, J. Tönshoff, and C. Morris. GraphBench: next-generation graph learning benchmarking. arXiv:2512.04475 (v5, May 2026), 2025.

A Witnesses and verification

Both blocks below are preimages of the 20-round SHA-256 compression function with the standard initial value, feed-forward retained and no padding constraint. The verifier is the standard-library script `code/verify_r19.py` from the repository; it shares no code with the attack.

```
# all-ones digest (the target of Zaikin, Constraints 2026)
python3 code/verify_r19.py --rounds 20 \
  --hash ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff \
  --words "0b0187bf b9aea692 b66effa5 087dca3a 0caea827 1d2f9916 739a224e a87c0eae
          7f9ef4a7 b318a7de a8848c61 a7a141a4 11ac114b 952299be 97aaa67f c6acfe57"

# solver-generated digest
python3 code/verify_r19.py --rounds 20 \
  --hash d962ca30635f9b74ac6c8c1243a1a9cf800e81bc05f1d2e40764c68c795f7388 \
  --words "a36f4238 f2c9204e b5b6653b 070401f7 5928e4f3 fe766be2 52026907 f7ee1812
          01344603 ea505012 86cbe6cb 6ce73d0b 5c91de6a 43355e3b ff3d5e88 c1ad0b54"
```

Each prints `result: OK`; with `--rounds 19` or `--rounds 21` each prints `result: FAIL`.

B Reproducibility

All code, logs, reviewer artefacts and this paper are in the repository at <https://github.com/kamb-code/sha256-r19-preimage>, under `code/` and `experiments/submask_2026-09-03/`.

- `code/submask_family.py`: the family, the triangular solve, the collapsed test, a table-free algebraic self-test of Propositions 1 and 2 (run with no arguments), and `--attack / --measure` modes.
- `code/build_sigma0_table.py`: builds the $\sigma_0(u) - u$ table (int32, sentinel -1 , largest root per value; about 215 core-seconds).
- `code/gpu_submask.py`: the PyTorch sweep with up to three root tables built on the device; `code/submask_triton.py`: the fused kernel with `--verify` (planted preimages) and `--bench`.

- `experiments/submask_2026-09-03/`: as-run scripts and logs with a sha256 manifest; `FLEET.txt` (hardware, target, seed, exposure, and recorded runtime); `NOTES.md` (corrections found by review); `R20_PREIMAGE_ALLONES.txt` and `R20_PREIMAGE.txt` (witnesses and provenance); `logs/r19_fixed_targets.log`, `logs/frame_search.log` and `logs/ones_obstruction.log` (the fixed-target runs, the context-family search of Table 4, and the all-ones obstruction test); `analysis/` (the corpus mining); `priority/` (the literature search and adjudication); `reviewers/` (separately written automated verification scripts, including three SHA-256 implementations written from the specification with constants derived from prime roots, and the full reports).
- Statistical method for nested thresholds: counts n_k of candidates whose residual has its low k bits zero are nested. Under the uniform null, n_{k+1} is conditionally $\text{Binomial}(n_k, \frac{1}{2})$. The manuscript reports conditional score summaries and multiplicity-adjusted individual transitions; it does not treat the random-denominator ratios as unconditionally independent.